

CRISP-DM: Data Preparation

Ilya Grigorev

Spring, 2026

1 Select Data

1.1 Inclusion / Exclusion Criteria

The [dataset](#) presents applications published on Steam in the last 30 years. Since business objectives are tied to finding game developers, all other types of applications should be excluded. Furthermore, in search of high relevance in the market, the analysis should be based on the last 10 years (2015-2025) of the game market. For a diverse look at each developer, the analysis is based on games with at least 1 sampled review from Steam. Finally, the complete set of developers is identified from the selected games: only developers with games that have reviews are selected.

The selection funnel for further data preparation is depicted in [Table 1](#)

Table 1: Data selection funnel.

Stage	Count
All applications	239,664
Games (<code>type='game'</code>)	150,279
Games released in 2015-2025	112,172
Games with at least one sampled review	89,518
All developers	101,226
Developers with at least one game having a sampled review	53,280

1.2 Selected Developer Metrics

Based on the Data Understanding phase, the relevant features that showed significant differences between the groups and were retained. [Table 2](#) and [Table 3](#) summarize these features and created metrics:

2 Clean Data

2.1 Pricing Unification

For consistent analysis, non-USD currencies are converted to USD.

Table 2: Review-derived features (micro aggregation).

Original field	Developer metric
<code>author_playtime_last_two_weeks</code>	average recent playtime (minutes)
<code>author_playtime_forever</code>	average total playtime (minutes)
<code>author_num_reviews</code>	average number of reviews written per player
<code>author_num_games_owned</code>	average number of games owned per player
<code>weighted_vote_score</code>	average helpfulness score

Table 3: Application-derived features.

Original field(s)	Derived developer metric
<code>appid</code>	<code>total_games</code> (number of games)
<code>is_free</code>	<code>percent_free</code> (free games / total games)
<code>mat_initial_price</code>	average initial price
<code>mat_discount_percent</code>	average discount percentage
<code>mat_achievement_count</code>	average achievement count
<code>release_date</code>	<code>years_past_first_release</code> (2025 - first release year)
<code>release_date + total_games</code>	<code>release_frequency</code> = <code>total_games</code> / (<code>years_past_first_release</code> + 1)
<code>supported_languages</code>	average number of languages per game
<code>genre_name</code>	<code>genre_count</code> (unique genres across portfolio)
<code>category_name</code>	<code>category_count</code> (unique categories across portfolio)

2.2 Noise in Prices

Some games demonstrate unrealistic prices (>\$200). Steam API lookup shows different values for these games. Therefore, using Steam API, these prices are replaced and converted to USD.

2.3 Games with no languages support

Some applications do not provide information on supported languages. The Steam pages show only local language support (1 supported language); therefore, the count should be set to 1.

2.4 Games with no price information

Many games lack price amounts information. 91.4% of these games are marked as free, and the initial price is imputed as 0.

The search on Steam showed that the other parts of the games are no longer listed; therefore, I propose to impute the initial price for these games using the median (robust to outliers).

2.5 Missing achievements count

Some games lack achievement information. The Steam API shows no results for these games (probably unlisted). I propose to impute missing values using the median (robust to outliers).

2.6 Missing genre count

If games of a developer have no genres, then clearly make the genre count zero.

2.7 Missing category count

If games of a developer have no categories, then clearly make category count zero.

2.8 Anomaly Report

All attributes have the correct ranges. The outliers are decided to be kept since the anomalies are not detected, and the top developers are overrepresented in the outlier regions for most features, which carries a strong signal to a model.

3 Construct Data

3.1 Target Variable

I constructed the target variable `is_top` as a binary flag indicating whether a developer has a partnership with at least one publisher from the list of top 100 publishers (by volume) identified during the Data Understanding phase. This label is used as the outcome in the subsequent modeling tasks.

3.2 Transformations

The prices in cents were transformed to align with the currency by division by 100. Additionally, for consistency, the discount percent values were re-scaled to the range $[0, 1]$ by division by 100.

3.3 Developer Metrics

The metrics and related attributes are thoroughly described in the [Table 2](#) and [Table 3](#).

3.4 Train-Test Split

The selected developers are first divided into sets of mature and emerging developers (recall that emerging developers are developers that have the first release within 3 years counting from 2025). The latter group is saved for the final business evaluation (counting developers with predicted probability > 0.7) and to assess the recall score on the top-partnered emerging developers, since the assigned labels for non-top developers may appear to be false negative (a developer team did not have enough time to demonstrate their accomplishments). The divided groups have 26,950 and 26,330 developers, respectively.

The resulting dataframe of mature developers is split into train set (used for modeling) and test set (used for model evaluation on the data mining success criteria) using stratification. The test part is set to 20% of the dataframe, resulting in 5,390 out of 26,950 records.

3.5 Clustering Analysis

Using K-means with the parameter $K = 5$, the train set developers were split into 5 groups. Note that the train set was transformed using the **StandardScaler** prior to clustering analysis.

In Table 4, I provide the interpretation of each cluster based on median values of metrics.

Table 4: Cluster characteristics and insights (k=5, trained on non-emerging developers).

Cluster	Name	Size	% of Total	Top %	Characteristics
0	Premium Discounters	1,025	4.8%	34.9%	Moderate portfolio, high discount (66%), moderate price, moderate languages. Highest top concentration.
2	High-Volume Diversified	2,726	12.6%	20.1%	Very large portfolio, extremely high release frequency (10 games/year), high genre/category diversity.
1	Hardcore Niche	559	2.6%	9.5%	Extremely dedicated player base (high reviewer stats), small portfolio, low price, few languages.
3	Average Indie	15,508	71.9%	7.0%	Small portfolio, low price, low diversity, low engagement. The typical indie developer.
4	Free-to-Play	1,742	8.1%	2.1%	All games free, zero price, low engagement, low diversity. Very low top proportion.

The cluster sizes and characteristics shown are based on the training subset (80% of mature developers, n=21,560).

Cluster analysis with 5 groups revealed a silhouette score of approximately 32%, which aligns with the data mining success criterion.

After scaling the training set with **StandardScaler**, K-means ($K = 5$) was fitted. The same scaler and K-means model (both fit on the training set) were applied to the test and emerging developers set to obtain cluster assignments.

4 Integrate Data

The review and application attributes are aggregated for each developer to create developer-level metrics (Table 2 and Table 3). Moreover, the resulting review related and application related metrics tables are merged (inner join) by the developer id. The final developer dataset contains 53,280 rows (all developers with games and reviews).

5 Format Data

Random shuffling was applied to train dataset. The cluster column was then replaced with 4 columns representing the flag for being from cluster 0, 1, 2, and 4: `cluster_premium_discounters`,

`cluster_hardcore_niche`, `cluster_high_volume_diversified`, `cluster_free_to_play`. The same encoding was applied to the test and emerging datasets to obtain the cluster indicator columns.

The resulting dataframes were stored as comma separated files.